

Rozgar ? Rubric Implementation Guide

Semester Project | Mobile Application Development
App: Rozgar Job Portal | Firebase Project: jobportal-b7092

How to Use This Document

This guide maps every semester rubric item to what it does, how it works, where it is implemented (file paths), and how to demo it during presentation/viva.

Live verification: Login as Admin ? System tab ? AdminDiagnosticsScreen

Rubric Score Summary

Rubric Item | Target Score | Status | Key Files

|---|---|---|---|

Complete App GUI | 5 | Implemented | lib/features/*/screens/, lib/core/widgets/

Firebase Authentication & DB | 4 | Implemented | lib/features/auth/,

lib/core/services/firestore_service.dart

Security: Encryption & Decryption | 4 | Implemented |

lib/core/security/aes_encryption_service.dart

App Architecture & Code Organization | 4 | Implemented | lib/features/, lib/core/

External REST API (non-Firebase) | 3 | Implemented | lib/core/services/job_repository.dart

Profiling | 3 | Implemented | lib/core/services/performance_service.dart

Logging and Debugging | 4 | Implemented | lib/core/utils/app_logger.dart

Notifications & Event Handling | 3 | Implemented | lib/core/services/fcm_service.dart

Background Tasks, Services, Threading | 3 | Implemented |

lib/core/services/workmanager_service.dart

Permissions | 4 | Implemented | lib/core/services/permission_service.dart

Advertisement / Monetization | 3 | Implemented | lib/core/services/ads_service.dart

Flutter Installation | 1 (env) | User machine | Run flutter doctor

1. Complete App GUI (Score: 5)

What It Does

Professional LinkedIn-style UI with glassmorphic AppBar, navigation drawer, animated job cards, role-based dashboards, shimmer loading, and responsive forms.

Where Implemented

Component | File Path

|---|---|

Glassmorphic AppBar | lib/core/widgets/glassmorphic_app_bar.dart

Navigation Drawer | lib/core/widgets/rozgar_drawer.dart

Job Cards (slide animation) | lib/core/widgets/sliding_job_card.dart

Shimmer skeleton loading | lib/core/widgets/shimmer_skeleton.dart

Skippable ad overlay UI | lib/core/widgets/skippable_ad_overlay.dart

App theme (light/dark) | lib/core/app_theme.dart, lib/core/providers/theme_provider.dart

User home (4 tabs) | lib/features/user/screens/user_home_screen.dart

Company home (4 tabs) | lib/features/company/screens/company_home_screen.dart

Admin home (5 tabs) | lib/features/admin/screens/admin_home_screen.dart

Login/Register UI | lib/features/auth/screens/login_screen.dart, register_screen.dart

Job creator wizard (3 steps) | lib/features/company/screens/company_job_creator_screen.dart

Application status tracker | lib/features/user/widgets/application_status_tracker.dart

How It Works

- Each role has a bottom navigation shell with tab pages
- GlassmorphicAppBar uses BackdropFilter blur for modern header
- RozgarShellBody adds top padding so content does not hide behind AppBar

- flutter_staggered_animations provides list entry animations
- cherry_toast shows success/error toasts

How to Demo

1. Login as User ? browse jobs feed with animated cards
2. Login as Company ? Post Job 3-step wizard
3. Toggle dark mode from AppBar icon
4. Open drawer from avatar tap

2. Firebase Authentication & Database (Score: 4)

What It Does

Email/password authentication with role-based profiles stored in Firestore. Real-time streams for jobs, applications, chats, and comments.

Where Implemented

Component | File Path

|---|---|

Firebase Auth service | lib/core/services/firebase_auth_service.dart

Auth repository (sign in/up/out) | lib/features/auth/provider/auth_repository.dart

Auth state notifier | lib/features/auth/provider/auth_provider.dart

Login screen + error mapping | lib/features/auth/screens/login_screen.dart

Register with role picker | lib/features/auth/screens/register_screen.dart

Firestore service | lib/core/services/firestore_service.dart

Security rules | firestore.rules

Firestore indexes | firestore.indexes.json

Firebase init | lib/main.dart, lib/firebase_options.dart

Firestore Collections

Collection | Purpose

|---|---|

users | User profiles (role, name, email, encrypted fields)

jobs | Premium company job listings

jobs/{id}/comments | Job comments (premium jobs only)

applications | Job applications

applications/{id}/status | Status history timeline

chats | Chat metadata (participants, last message)

chats/{id}/messages | Chat messages

How It Works

1. User registers ? Firebase Auth creates account ? Firestore users/{uid} document created with role
2. authStateProvider streams Firebase auth state
3. GoRouter redirects to role home (/user, /company, /admin)
4. All queries require signed-in user per firestore.rules

Login Error Messages

Firebase Code | User Message

|---|---|

invalid-credential | The email or password is incorrect

wrong-password | Incorrect password

user-not-found | No account exists with this email

too-many-requests | Too many failed attempts

How to Demo

1. Register as Job Seeker / Recruiter / Admin
2. Login with wrong password ? see red error banner

3. Open Firebase Console ? show users, jobs, applications collections
4. Show deployed rules: firebase deploy --only firestore:rules

3. Security: Encryption & Decryption (Score: 4)

What It Does

AES-256 CTR encryption for sensitive PII before writing to Firestore. Encryption key stored in Flutter Secure Storage.

Where Implemented

Component | File Path

|---|---|

AES encryption service | lib/core/security/aes_encryption_service.dart

Secure key storage | lib/core/security/secure_storage_service.dart

Sensitive field list | lib/features/auth/constants/auth_constants.dart

Encrypt on register | lib/features/auth/provider/auth_repository.dart

Encrypt profile salary/phone | lib/features/user/provider/user_profile_provider.dart

Key init on startup | lib/main.dart (aesEncryptionProvider.initialize)

Encrypted Fields

- phone (user registration/profile)
- salary (user profile, job salary)
- identityDocument (if used)

How It Works

1. On first launch, random 256-bit key generated and saved in Secure Storage
2. Before Firestore write: encryption.encrypt(plaintext) ? ciphertext string
3. On read: encryption.decrypt(ciphertext) ? original value
4. Admin diagnostics runs round-trip test: encryption("diagnostics") ? decrypt ? verify

How to Demo

1. Edit user profile phone number ? save
2. Open Firebase Console ? users collection ? phone field shows encrypted text (not plain number)
3. Admin ? System tab ? "AES-256 CTR Service" shows PASS

4. App Architecture & Code Organization (Score: 4)

What It Does

Feature-first clean architecture separating shared infrastructure from role-specific features.

Folder Structure

```
lib/
??? main.dart      # Bootstrap
??? app.dart       # MaterialApp.router
??? core/          # Shared infrastructure
?   ??? constants/ # App, API, route, ads constants
?   ??? models/    # JobModel, ChatMessage, UserRole
?   ??? network/   # Dio HTTP client
?   ??? providers/  # Riverpod DI
?   ??? router/    # GoRouter + auth guards
?   ??? security/   # Encryption
?   ??? services/   # Firebase, Cloudinary, ads, etc.
?   ??? utils/     # Logger, Result, extensions
?   ??? widgets/   # Shared UI components
??? features/
```

??? auth/	# Login, register, splash
??? user/	# Job seeker module
??? company/	# Recruiter module
??? admin/	# Admin module

Design Patterns Used

Pattern | Where

|---|---|

Repository | auth_repository.dart, job_repository.dart, application_provider.dart

Provider DI | lib/core/providers/core_providers.dart

Result type (Success/Failure) | lib/core/utis/result.dart

Feature modules per role | lib/features/user/, company/, admin/

Shared widgets | lib/core/widgets/

How to Demo

Explain folder structure in IDE. Show one feature module has: constants/, model/, provider/, screens/, widgets/

5. External REST API Integration (Score: 3)

What It Does

Fetches jobs from JSONPlaceholder REST API and merges with Firestore premium jobs in a single hybrid feed.

Where Implemented

Component | File Path

|---|---|

Dio HTTP client | lib/core/network/dio_client.dart

API endpoint constant | lib/core/constants/api_constants.dart

Hybrid job repository | lib/core/services/job_repository.dart

External job model parser | lib/core/models/job_model.dart (fromExternalApi)

User jobs provider | lib/features/user/provider/user_jobs_provider.dart

Jobs screen | lib/features/user/screens/user_jobs_screen.dart

API Endpoint

<https://jsonplaceholder.typicode.com/posts>

How It Works

1. fetchHybridJobs() runs two fetches in parallel (resilient ? one can fail)
2. Firestore jobs marked isPremium: true, sorted first
3. External jobs get IDs like ext_1, ext_2 with source: external
4. Combined list shown on User Jobs tab

How to Demo

1. User ? Jobs tab ? scroll feed
2. Premium jobs (company-posted) appear at top with banner images
3. External jobs appear below (from REST API)
4. Check logs: AppLogger.network or Admin ? System ? debug logs

6. Profiling (Score: 3)

What It Does

Firebase Performance Monitoring custom traces measure operation duration for auth, job fetching, and uploads.

Where Implemented

Component | File Path

|---|---|

Performance service | lib/core/services/performance_service.dart
Auth sign-in trace | lib/features/auth/provider/auth_repository.dart
Job fetch trace | lib/core/services/job_repository.dart
Cloudinary upload trace | lib/core/services/cloudinary_service.dart

Trace Names

Trace | Trigger

|---|---|

auth_sign_in | User logs in
fetch_hybrid_jobs | Jobs feed loads
post_premium_job | Company publishes job
cloudinary_upload_* | Image/resume upload

How It Works

```
await _performance.trace('auth_sign_in', () async {  
  // operation code  
});
```

Firebase Performance SDK records start/stop timestamps automatically.

How to Demo

1. Use app normally (login, browse jobs, upload image)
2. Open Firebase Console ? Performance ? Custom traces
3. Show trace durations after a few minutes of data collection

7. Logging and Debugging (Score: 4)

What It Does

Structured application logging with levels, plus in-memory log store viewable in Admin diagnostics screen.

Where Implemented

Component | File Path

|---|---|

AppLogger | lib/core/utils/app_logger.dart
Debug log store (ring buffer) | lib/core/utils/debug_log_store.dart
Admin log viewer | lib/features/admin/screens/admin_diagnostics_screen.dart
Diagnostics service | lib/core/services/app_diagnostics_service.dart

Log Levels

Level | Method | Use Case

|---|---|---|

DEBUG | AppLogger.debug() | Development details
INFO | AppLogger.info() | Normal operations
WARN | AppLogger.warning() | Permission denied, retries
ERROR | AppLogger.error() | Recoverable errors
SEVERE | AppLogger.severe() | Critical failures with stack trace
AUTH | AppLogger.auth() | Authentication events
NETWORK | AppLogger.network() | API calls

How It Works

1. Every log call writes to DebugLogStore (max 100 entries)
2. In debug mode, also prints to console via logger package
3. Admin ? System tab shows last 20 logs with timestamps and icons

How to Demo

1. Use app (login, apply, upload) to generate logs

2. Login as Admin ? System tab ? scroll to "Live Debug Logs"
3. Run flutter run ? show terminal console output

8. Notifications & Event Handling (Score: 3)

What It Does

Firebase Cloud Messaging (FCM) for push notifications + local notifications for application submit and foreground FCM messages.

Where Implemented

Component | File Path

|---|---|

FCM service | lib/core/services/fcm_service.dart

Local notifications | lib/core/services/local_notification_service.dart

FCM background handler | lib/main.dart

Apply notification trigger | lib/features/user/provider/application_provider.dart

Android POST_NOTIFICATIONS | android/app/src/main/AndroidManifest.xml

Notification Channels (Android)

Channel | Purpose

|---|---|

messages | Chat notifications

applications | Application status alerts

How It Works

1. On startup: request notification permission, initialize FCM, get device token
2. Foreground FCM message ? converted to local notification
3. On job apply: showApplicationNotification() fires immediately
4. Background handler registered in main.dart

How to Demo

1. Apply to a job ? local notification appears
2. Send FCM test from Firebase Console ? notification on device
3. Admin ? System ? "FCM + Local Notifications" shows PASS

9. Background Tasks, Services & Threading (Score: 3)

What It Does

WorkManager for periodic background sync and one-off tasks; Dart Isolates for CPU-heavy work off the main UI thread.

Where Implemented

Component | File Path

|---|---|

WorkManager service | lib/core/services/workmanager_service.dart

Isolate service | lib/core/services/isolate_service.dart

WorkManager init | lib/main.dart

Apply task trigger | lib/features/user/provider/application_provider.dart

Android permissions | android/app/src/main/AndroidManifest.xml (WAKE_LOCK, RECEIVE_BOOT_COMPLETED)

WorkManager Tasks

Task | Type | Purpose

|---|---|---|

rozgar_decrypt_task | One-off | Post-application background processing

rozgar_sync_task | Periodic (6h) | Background data sync

Isolate Operations

Operation | Purpose

|---|---|

JSON parsing | Offload large API response parsing

Decryption | Offload AES decrypt from UI thread

How It Works

1. Workmanager.initialize() in bootstrap with callback dispatcher
2. On application submit: scheduleDecryptTask() enqueued
3. IsolateService spawns isolate for heavy computation
4. Mobile only (skipped on web)

How to Demo

1. Submit job application ? WorkManager task scheduled (check logs)
2. Admin ? System ? WorkManager + Isolates show PASS
3. Explain isolates prevent UI jank during heavy operations

10. Permissions (Score: 4)

What It Does

Runtime permission requests for camera, gallery, and notifications with rationale dialogs and settings fallback.

Where Implemented

Component | File Path

|---|---|

Permission service | lib/core/services/permission_service.dart

Media permission helper (UI) | lib/core/widgets/media_permission_helper.dart

Profile photo picker | lib/core/widgets/profile_avatar_picker.dart

Job banner upload | lib/features/company/screens/company_job_creator_screen.dart

Resume upload | lib/features/user/widgets/job_application_sheet.dart

Android manifest | android/app/src/main/AndroidManifest.xml

iOS Info.plist | ios/Runner/Info.plist

Permission Flow

1. User taps upload/camera
2. Rationale dialog explains why permission is needed
3. System dialog shows Allow / Deny (Allow this time on Android)
4. If permanently denied ? Open Settings dialog

Permissions Requested

Permission | When

|---|---|

Camera | Profile photo, job banner (camera option)

Photos/Gallery | Gallery upload, resume

Notifications | App startup

Storage (legacy Android) | Fallback for older devices

How to Demo

1. Tap profile camera icon ? see rationale ? system permission prompt
2. Deny permission ? see error toast
3. Deny permanently ? see "Open Settings" dialog
4. Admin ? System ? permission status checks

11. Advertisement / Monetization (Score: 3)

What It Does

Google Mobile Ads with banner ads on home screens and skippable full-screen interstitial overlay.

Where Implemented

Component | File Path

|---|---|

Ads service | lib/core/services/ads_service.dart

Banner ad widget | lib/core/widgets/banner_ad_widget.dart

Skippable overlay | lib/core/widgets/skippable_ad_overlay.dart

Ad unit IDs (test) | lib/core/constants/ads_constants.dart

User home scheduling | lib/features/user/screens/user_home_screen.dart

AdMob app ID | android/app/src/main/AndroidManifest.xml

Ad Behavior

Ad Type | Behavior

|---|---|

Banner | Always visible above bottom navigation

Interstitial overlay | Appears after ~2.5 minutes; 5-second skip countdown; X close button

How to Demo

1. Open User home ? banner ad at bottom
2. Wait 2.5 minutes ? full-screen ad with skip timer
3. Tap X or wait for skip ? ad closes
4. Test device ID whitelisted in ads_constants.dart

12. Flutter Installation (Score: 1 ? Environment)

What It Does

Development environment setup on your machine.

Commands to Verify

flutter doctor

flutter pub get

flutter run

dart run flutter_launcher_icons

Project Builds On

- Android (primary target)
- iOS (with Info.plist permissions)
- Web (limited ? no ads/WorkManager)

Quick Demo Checklist (Presentation Order)

| Action | Rubric Item

|---|---|---|

- 1 | Register + Login (show error on wrong password) | Auth
- 2 | Browse jobs (Firestore + REST hybrid) | REST API, GUI
- 3 | Apply to job ? notification appears | Notifications
- 4 | Check application status tracker | Firebase DB, GUI
- 5 | Message recruiter from job detail | Chat, Firebase
- 6 | Company: Post job with banner upload | GUI, Cloudinary, Permissions
- 7 | Upload profile photo (permission flow) | Permissions
- 8 | Wait for skippable ad | Ads
- 9 | Admin ? System diagnostics | All rubric verification
- 10 | Firebase Console ? Performance traces | Profiling
- 11 | Firebase Console ? encrypted phone field | Security

